

NLP Advanced Project: Deceptive Opinion Spam Detection

Objectives

1. **Exploratory Data Analysis (EDA):** Characterize 'truthful' vs 'deceptive' reviews.
2. **Switch to PyTorch:** Implement a robust Bi-LSTM model.
3. **K-Fold Cross-Validation:** Address the small dataset size (1600 samples).
4. **BERT Comparison:** Fine-tune DistilBERT and compare it with the Bi-LSTM.
5. **Overfitting/Underfitting Analysis:** Visualize and interpret training curves.
6. **Interactive Interface:** Use Gradio for real-time review classification.

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import KFold, train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accu
from tqdm.auto import tqdm
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

```
Using device: cuda
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

1. Data Loading and Initial Exploration

```
# Download the dataset
path = kagglehub.dataset_download("rtatman/deceptive-opinion-spam-corpus")
csv_path = os.path.join(path, "deceptive-opinion.csv")
df = pd.read_csv(csv_path)

# Map labels: truthful -> 0, deceptive -> 1
df['label'] = df['deceptive'].map({'truthful': 0, 'deceptive': 1})

print(f"Dataset shape: {df.shape}")
df.head()
```

Using Colab cache for faster access to the 'deceptive-opinion-spam-corpus'
Dataset shape: (1600, 6)

	deceptive	hotel	polarity	source	text	label
0	truthful	conrad	positive	TripAdvisor	We stayed for a one night getaway with family ...	0
1	truthful	hyatt	positive	TripAdvisor	Triple A rate with upgrade to view room was le...	0
2	truthful	hyatt	positive	TripAdvisor	This comes a little late as I'm finally catchi	0

Étapes suivantes :

[Générer du code avec df](#)

[New interactive sheet](#)

2. Characterizing True vs Fake Reviews

Let's analyze the text features to see if there are recognizable patterns in deceptive reviews.

```
def get_text_features(text):
    words = word_tokenize(text.lower())
    return {
        'length': len(text),
        'word_count': len(words),
        'punct_count': sum([1 for char in text if char in string.punctuation]),
        'stopword_count': sum([1 for word in words if word in stopwords.words('english')])
    }

features_df = df['text'].apply(lambda x: pd.Series(get_text_features(x)))
df_stats = pd.concat([df[['label']], features_df], axis=1)

# Comparison
comparison = df_stats.groupby('label').mean()
comparison.index = ['Truthful', 'Deceptive']
display(comparison)
```

```
display(comparison)
```

	length	word_count	punct_count	stopword_count
Truthful	821.0150	171.5275	23.8700	74.9525
Deceptive	791.7675	163.4050	18.0225	75.8025



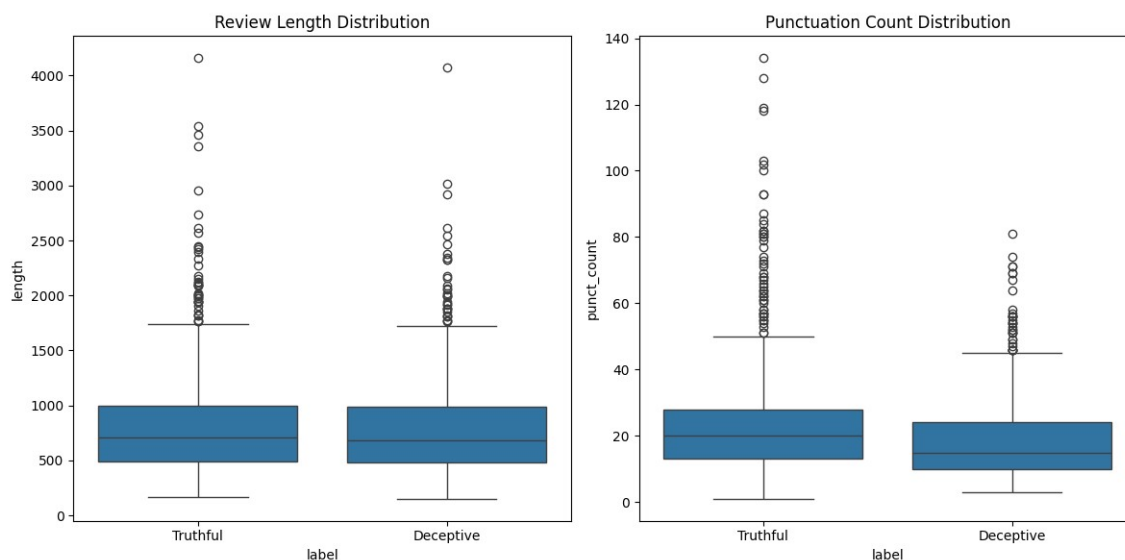
Étapes suivantes :

[Générer du code avec comparaison](#)[New interactive sheet](#)

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
sns.boxplot(x='label', y='length', data=df_stats)
plt.title('Review Length Distribution')
plt.xticks([0, 1], ['Truthful', 'Deceptive'])

plt.subplot(1, 2, 2)
sns.boxplot(x='label', y='punct_count', data=df_stats)
plt.title('Punctuation Count Distribution')
plt.xticks([0, 1], ['Truthful', 'Deceptive'])

plt.tight_layout()
plt.show()
```



```

# Camembert pour visualiser la proportion de vrais et faux avis
plt.figure(figsize=(15, 5))

# Comptage des labels
label_counts = df['label'].value_counts()
print("Distribution des classes:")
print(f"Truthful (réels): {label_counts[0]} avis")
print(f"Deceptive (trompeurs): {label_counts[1]} avis")

plt.subplot(1, 3, 1)
# Camembert
colors = ['#4ECDC4', '#FF6B6B'] # Vert turquoise pour truthful, rouge pour
plt.pie(label_counts.values,
        labels=['Truthful (Réels)', 'Deceptive (Trompeurs)'],
        colors=colors,
        autopct='%1.1f%%',
        startangle=90,
        explode=(0.05, 0.05), # Légère séparation
        shadow=True,
        textprops={'fontsize': 12, 'fontweight': 'bold'})
plt.title('Distribution des classes', fontsize=14, fontweight='bold')

# Ajouter un camembert plus détaillé avec les pourcentages par hôtel
plt.subplot(1, 3, 2)
hotel_dist = df.groupby('hotel')['label'].value_counts().unstack().fillna(0)
hotel_dist.columns = ['Truthful', 'Deceptive']
hotel_dist.plot(kind='bar', stacked=True, color=colors, ax=plt.gca())
plt.title('Distribution par hôtel', fontsize=14, fontweight='bold')
plt.xlabel('Hôtel')
plt.ylabel('Nombre d\'avis')
plt.legend(title='Type')
plt.xticks(rotation=45, ha='right')

# Camembert pour la proportion par source
plt.subplot(1, 3, 3)
source_counts = df['source'].value_counts()
plt.pie(source_counts.values,
        labels=source_counts.index,
        autopct='%1.1f%%',
        startangle=90,
        colors=['#95E1D3', '#FFD93D', '#4ECDC4', '#FF6B6B'])
plt.title('Distribution par source', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()

# Statistiques détaillées
print("\n" + "="*50)
print("📊 STATISTIQUES DÉTAILLÉES")
print("="*50)

```

```

total = len(df)
truthful_pct = (label_counts[0] / total) * 100
deceptive_pct = (label_counts[1] / total) * 100

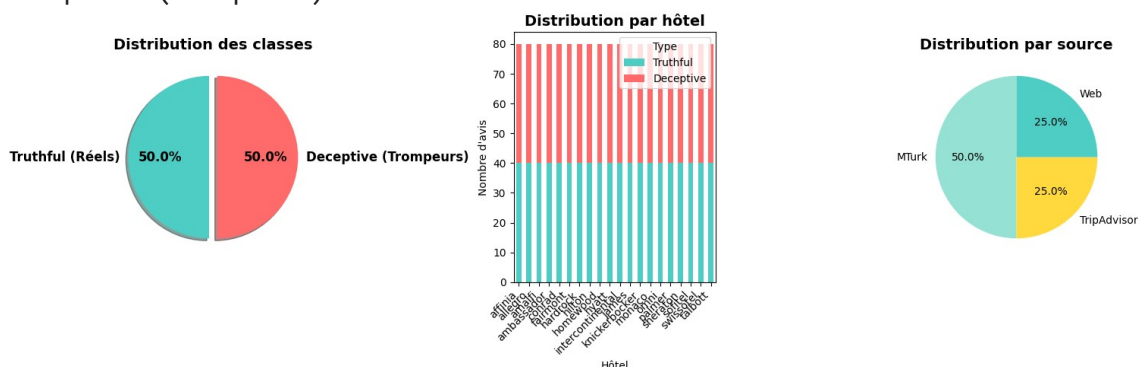
print(f"\n📊 Ratio global:")
print(f" • Truthful (Réels): {label_counts[0]} avis ({truthful_pct:.1f}%)"
print(f" • Deceptive (Faux): {label_counts[1]} avis ({deceptive_pct:.1f}%)"

print(f"\n🏨 Distribution par hôtel:")
for hotel in df['hotel'].unique():
    hotel_data = df[df['hotel'] == hotel]
    truthful = len(hotel_data[hotel_data['label'] == 0])
    deceptive = len(hotel_data[hotel_data['label'] == 1])
    total_hotel = len(hotel_data)
    print(f" • {hotel}: {truthful} truth ({truthful/total_hotel*100:.0f}%)

print(f"\n📄 Distribution par source:")
for source in df['source'].unique():
    source_data = df[df['source'] == source]
    print(f" • {source}: {len(source_data)} avis")

```

Distribution des classes:
 Truthful (réels): 800 avis
 Deceptive (trompeurs): 800 avis



STATISTIQUES DÉTAILLÉES

📊 Ratio global:

- Truthful (Réels): 800 avis (50.0%)
- Deceptive (Faux): 800 avis (50.0%)

🏨 Distribution par hôtel:

- conrad: 40 truth (50%) / 40 dec (50%)
- hyatt: 40 truth (50%) / 40 dec (50%)
- omni: 40 truth (50%) / 40 dec (50%)
- fairmont: 40 truth (50%) / 40 dec (50%)
- sheraton: 40 truth (50%) / 40 dec (50%)
- knickerbocker: 40 truth (50%) / 40 dec (50%)
- homewood: 40 truth (50%) / 40 dec (50%)
- swissotel: 40 truth (50%) / 40 dec (50%)
- ambassador: 40 truth (50%) / 40 dec (50%)
- affinia: 40 truth (50%) / 40 dec (50%)
- hardrock: 40 truth (50%) / 40 dec (50%)
- talbott: 40 truth (50%) / 40 dec (50%)

- hilton: 40 truth (50%) / 40 dec (50%)
- james: 40 truth (50%) / 40 dec (50%)
- monaco: 40 truth (50%) / 40 dec (50%)
- sofitel: 40 truth (50%) / 40 dec (50%)
- palmer: 40 truth (50%) / 40 dec (50%)
- intercontinental: 40 truth (50%) / 40 dec (50%)
- allegro: 40 truth (50%) / 40 dec (50%)
- amalfi: 40 truth (50%) / 40 dec (50%)



Distribution par source:

- TripAdvisor: 400 avis
- MTurk: 800 avis
- Web: 400 avis

Commencez à coder ou à [générer](#) avec l'IA.

✓ 3. PyTorch Bi-LSTM Implementation

We'll start by preparing the vocabulary and the dataset pipeline.

```
from collections import Counter

def clean_text(text):
    text = text.lower()
    text = re.sub(f"[{re.escape(string.punctuation)}]", "", text)
    return text

# Vocabulary Building
all_text = " ".join(df["text"].apply(clean_text))
words = all_text.split()
vocab = Counter(words)
vocab = sorted(vocab, key=vocab.get, reverse=True)
vocab_to_int = {word: ii + 1 for ii, word in enumerate(vocab[:10000])} #
vocab_to_int["<PAD>"] = 0
```

vocab

```
['the',  
'and',  
'to',  
'a',  
'i',  
'was',  
'in',  
'of',  
'hotel',  
'we',  
'for',  
'room',  
'it',  
'at',  
'my',  
'is',  
'that',  
'with',  
'were',  
'not',  
'had',  
'this',  
'on',  
'chicago',  
'but',  
'our',  
'they',  
'very',  
'stay',  
'have',  
'there',  
'you',  
'be',  
'as',  
'from',  
'when',  
'all',  
'would',  
'great',  
'so',  
'staff',  
'me',  
'are',  
'service',  
'one',  
'rooms',  
'stayed',  
'an',  
'out',  
'no',  
'up',  
'us',  
'if',  
'like',
```

```
'get',
'will',
'night',
'just',
```

Commencez à coder ou à [générer](#) avec l'IA.

```
def tokenize_text(text, max_len=200):
    text = clean_text(text)
    tokens = [vocab_to_int.get(word, 0) for word in text.split()][:max_len]
    # Padding
    if len(tokens) < max_len:
        tokens = tokens + [0] * (max_len - len(tokens))
    return tokens

df["tokenized"] = df["text"].apply(lambda x: tokenize_text(x))
print("Tokenization complete.")
```

Tokenization complete.

```
class ReviewDataset(Dataset):
    def __init__(self, tokenized_texts, labels):
        self.data = tokenized_texts
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return torch.tensor(self.data[idx]), torch.tensor(self.labels[idx])
```

```
class BiLSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
                 super(BiLSTMClassifier, self).__init__())
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim,
                             hidden_dim,
                             num_layers=n_layers,
                             bidirectional=True,
                             dropout=dropout,
                             batch_first=True)
        self.fc = nn.Linear(hidden_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x shape: (batch_size, seq_len)
        embedded = self.dropout(self.embedding(x))
        # embedded shape: (batch_size, seq_len, embedding_dim)
        lstm_out, (hidden, cell) = self.lstm(embedded)
        # hidden shape: (num_layers * 2, batch_size, hidden_dim)

        # Concat the final forward and backward hidden states
```

```
# Concat the final forward and backward hidden states
hidden = self.dropout(torch.cat((hidden[-2,:,:], hidden[-1,:,:])),
# hidden shape: (batch_size, hidden_dim * 2)

out = self.fc(hidden)
return self.sigmoid(out)
```

```
def train_model(model, train_loader, val_loader, criterion, optimizer, ep
history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_ac
best_val_acc = 0

for epoch in range(epochs):
    model.train()
    train_losses = []
    correct_train = 0
    total_train = 0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        output = model(inputs).squeeze()
        loss = criterion(output, labels)
        loss.backward()
        optimizer.step()

        train_losses.append(loss.item())
        preds = (output > 0.5).float()
        correct_train += (preds == labels).sum().item()
        total_train += labels.size(0)

    model.eval()
    val_losses = []
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            output = model(inputs).squeeze()
            loss = criterion(output, labels)
            val_losses.append(loss.item())
            preds = (output > 0.5).float()
            correct_val += (preds == labels).sum().item()
            total_val += labels.size(0)

    history["train_loss"].append(np.mean(train_losses))
    history["val_loss"].append(np.mean(val_losses))
    history["train_acc"].append(correct_train / total_train)
    history["val_acc"].append(correct_val / total_val)

    if history["val_acc"][-1] > best_val_acc:
        best_val_acc = history["val_acc"][-1]
        torch.save(model.state_dict(), "best_lstm.pt")
```

```
return history
```

```

kf = KFold(n_splits=5, shuffle=True, random_state=42)
fold_results = []

X = np.array(df["tokenized"].tolist())
y = df["label"].values

print("Starting K-Fold Cross Validation for Bi-LSTM...")
for fold, (train_idx, val_idx) in enumerate(kf.split(X)):
    print(f"Fold {fold+1}/5")

    train_set = ReviewDataset(X[train_idx], y[train_idx])
    val_set = ReviewDataset(X[val_idx], y[val_idx])

    train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
    val_loader = DataLoader(val_set, batch_size=32)

    model = BiLSTMClassifier(len(vocab_to_int), 100, 128, 1, 2, 0.5).to(dev)
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    history = train_model(model, train_loader, val_loader, criterion, optim)
    fold_results.append(history)
    print(f"Best Val Acc: {max(history["val_acc"]):.4f}")

```

```

Starting K-Fold Cross Validation for Bi-LSTM...
Fold 1/5
Best Val Acc: 0.7688
Fold 2/5
Best Val Acc: 0.7719
Fold 3/5
Best Val Acc: 0.7875
Fold 4/5
Best Val Acc: 0.8000
Fold 5/5
Best Val Acc: 0.7844

```

```

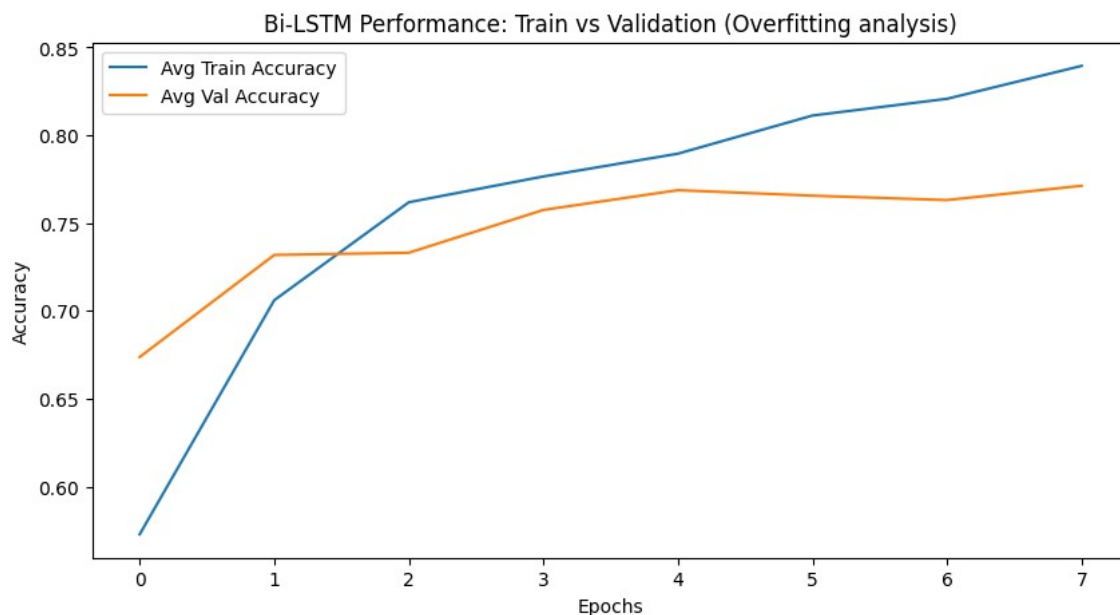
def plot_comparison(fold_results):
    avg_train_acc = np.mean([res["train_acc"] for res in fold_results], axis=0)
    avg_val_acc = np.mean([res["val_acc"] for res in fold_results], axis=0)

    plt.figure(figsize=(10, 5))
    plt.plot(avg_train_acc, label="Avg Train Accuracy")
    plt.plot(avg_val_acc, label="Avg Val Accuracy")
    plt.title("Bi-LSTM Performance: Train vs Validation (Overfitting analysis)")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.show()

    # If train accuracy >> val accuracy, we have overfitting.
    # If both are low, we have underfitting.

```

```
plot_comparison(fold_results)
```



```
def evaluate_lstm_model(model, data_loader, dataset_name="Validation"):
    """Évalue le modèle Bi-LSTM et retourne les métriques détaillées"""
    model.eval()
    all_preds = []
    all_labels = []
    all_probs = []

    with torch.no_grad():
        for inputs, labels in data_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs).squeeze()

            probs = outputs.cpu().numpy()
            preds = (outputs > 0.5).float().cpu().numpy()

            all_preds.extend(preds)
            all_labels.extend(labels.cpu().numpy())
            all_probs.extend(probs)

    # Calcul des métriques
    accuracy = accuracy_score(all_labels, all_preds)
    from sklearn.metrics import roc_auc_score # Import roc_auc_score here
    auc_roc = roc_auc_score(all_labels, all_probs)

    print(f"\n--- Métriques {dataset_name} - Bi-LSTM ---")
```

```

    print(f"====> Métriques {dataset_name} - Bi-LSTM ->====> ")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"AUC-ROC: {auc_roc:.4f}")
    print("\nClassification Report:")
    print(classification_report(all_labels, all_preds,
                                target_names=['Truthful (0)', 'Deceptive (1)']

    # Matrice de confusion
    cm = confusion_matrix(all_labels, all_preds)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges',
                xticklabels=['Truthful', 'Deceptive'],
                yticklabels=['Truthful', 'Deceptive'])
    plt.title(f'Matrice de Confusion - Bi-LSTM ({dataset_name})')
    plt.ylabel('Vraie étiquette')
    plt.xlabel('Prédiction')
    plt.show()

    return {
        'accuracy': accuracy,
        'auc_roc': auc_roc,
        'predictions': all_preds,
        'labels': all_labels,
        'probabilities': all_probs,
        'confusion_matrix': cm
    }

# Pour le Bi-LSTM, nous devons refaire un split train/val/test
# ou utiliser le dernier fold pour l'analyse détaillée

# 1. D'abord, créons un split train/val/test
X_train_text, X_temp_text, y_train, y_temp = train_test_split(
    df['text'], df['label'], test_size=0.3, random_state=42, stratify=df['l
)
X_val_text, X_test_text, y_val, y_test = train_test_split(
    X_temp_text, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

# 2. Tokenisation
X_train_tokens = [tokenize_text(text) for text in X_train_text]
X_val_tokens = [tokenize_text(text) for text in X_val_text]
X_test_tokens = [tokenize_text(text) for text in X_test_text]

# 3. Création des datasets
train_dataset_lstm = ReviewDataset(X_train_tokens, y_train.values)
val_dataset_lstm = ReviewDataset(X_val_tokens, y_val.values)
test_dataset_lstm = ReviewDataset(X_test_tokens, y_test.values)

train_loader_lstm = DataLoader(train_dataset_lstm, batch_size=32, shuffle=True)
val_loader_lstm = DataLoader(val_dataset_lstm, batch_size=32)
test_loader_lstm = DataLoader(test_dataset_lstm, batch_size=32)

# 4. Entraînement d'un modèle final (ou chargement du meilleur)
print("\n" + "=" * 50)
print("FINTRAÎNEMENT DU MODÈLE BI-LSTM FINAL")

```

```

print( ENTRAÎNEMENT DU MODÈLE BI-LSTM FINAL ,
print("=" * 50)

model_lstm_final = BiLSTMClassifier(len(vocab_to_int), 100, 128, 1, 2, 0.5)
criterion = nn.BCELoss()
optimizer = optim.Adam(model_lstm_final.parameters(), lr=0.001)

# Si vous avez sauvegardé le meilleur modèle pendant la CV, chargez-le
if os.path.exists("best_lstm.pt"):
    model_lstm_final.load_state_dict(torch.load("best_lstm.pt"))
    print("Modèle chargé depuis best_lstm.pt")
else:
    # Sinon, réentraînez
    history_final = train_model(model_lstm_final, train_loader_lstm, val_loader_lstm,
                                criterion, optimizer, epochs=8)

# 5. Évaluation
print("\n" + "=" * 50)
print("ÉVALUATION BI-LSTM")
print("=" * 50)

train_metrics_lstm = evaluate_lstm_model(model_lstm_final, train_loader_lstm)
val_metrics_lstm = evaluate_lstm_model(model_lstm_final, val_loader_lstm, "validation")
test_metrics_lstm = evaluate_lstm_model(model_lstm_final, test_loader_lstm, "test")

# Résumé comparatif
print("\n" + "=" * 50)
print("RÉSUMÉ BI-LSTM")
print("=" * 50)
print(f"{'Dataset':<15} {'Accuracy':<12} {'AUC-ROC':<12}")
print("-" * 39)
print(f"{'Train':<15} {train_metrics_lstm['accuracy']:.4f} {train_metrics_lstm['auc-roc']:.4f}")
print(f"{'Validation':<15} {val_metrics_lstm['accuracy']:.4f} {val_metrics_lstm['auc-roc']:.4f}")
print(f"{'Test':<15} {test_metrics_lstm['accuracy']:.4f} {test_metrics_lstm['auc-roc']:.4f}")

```

```

=====
ENTRAÎNEMENT DU MODÈLE BI-LSTM FINAL
=====

```

```
Modèle chargé depuis best_lstm.pt
```

```

=====
ÉVALUATION BI-LSTM
=====

```

```
=== Métriques Train - Bi-LSTM ===
```

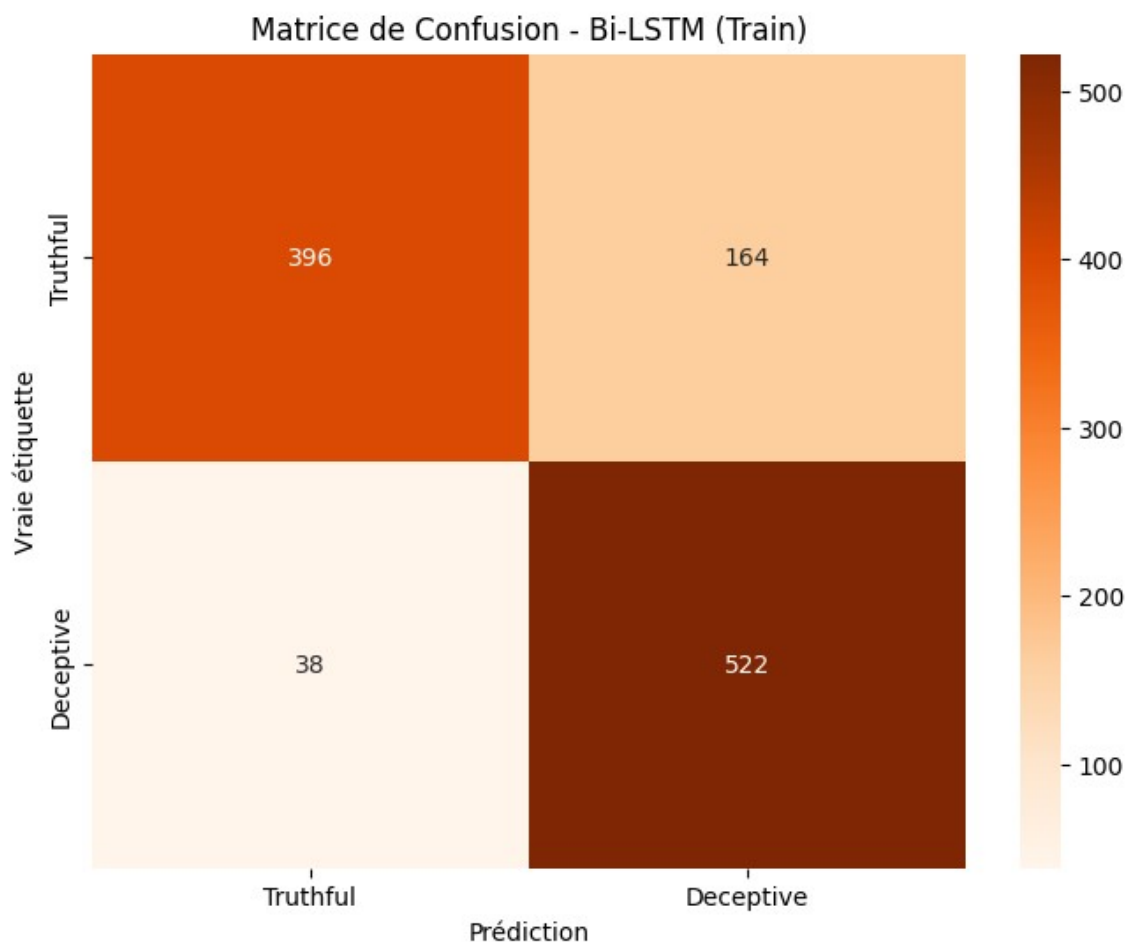
```
Accuracy: 0.8196
```

```
AUC-ROC: 0.9268
```

```
Classification Report:
```

	precision	recall	f1-score	support
Truthful (0)	0.91	0.71	0.80	560
Deceptive (1)	0.76	0.93	0.84	560
accuracy			0.82	1120

macro avg	0.84	0.82	0.82	1120
weighted avg	0.84	0.82	0.82	1120



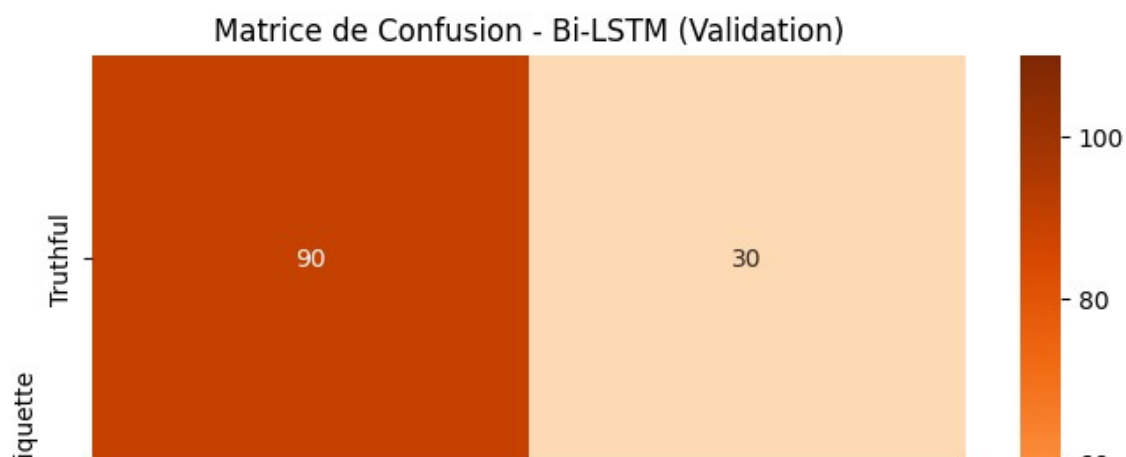
=== Métriques Validation - Bi-LSTM ===

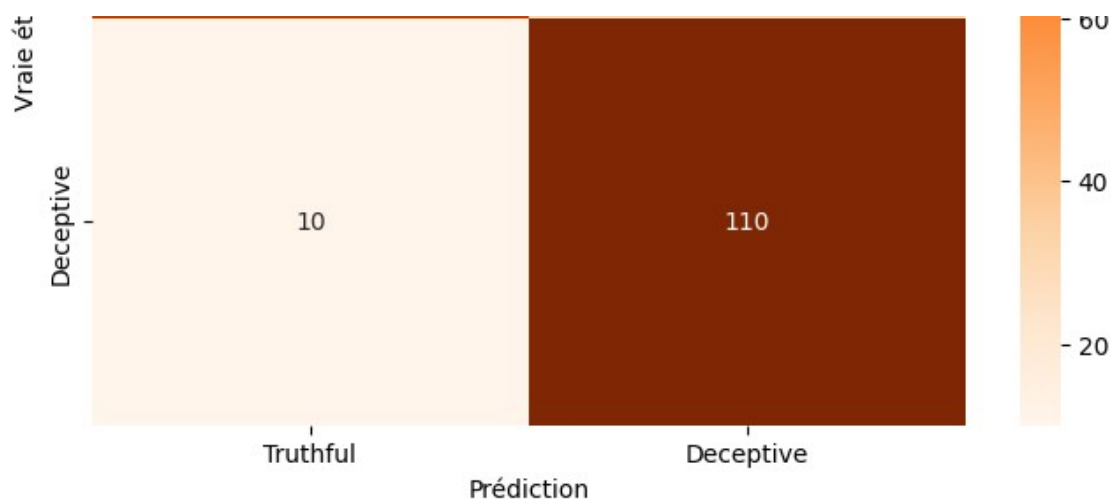
Accuracy: 0.8333

AUC-ROC: 0.9313

Classification Report:

	precision	recall	f1-score	support
Truthful (0)	0.90	0.75	0.82	120
Deceptive (1)	0.79	0.92	0.85	120
accuracy			0.83	240
macro avg	0.84	0.83	0.83	240
weighted avg	0.84	0.83	0.83	240





=== Métriques Test - Bi-LSTM ===

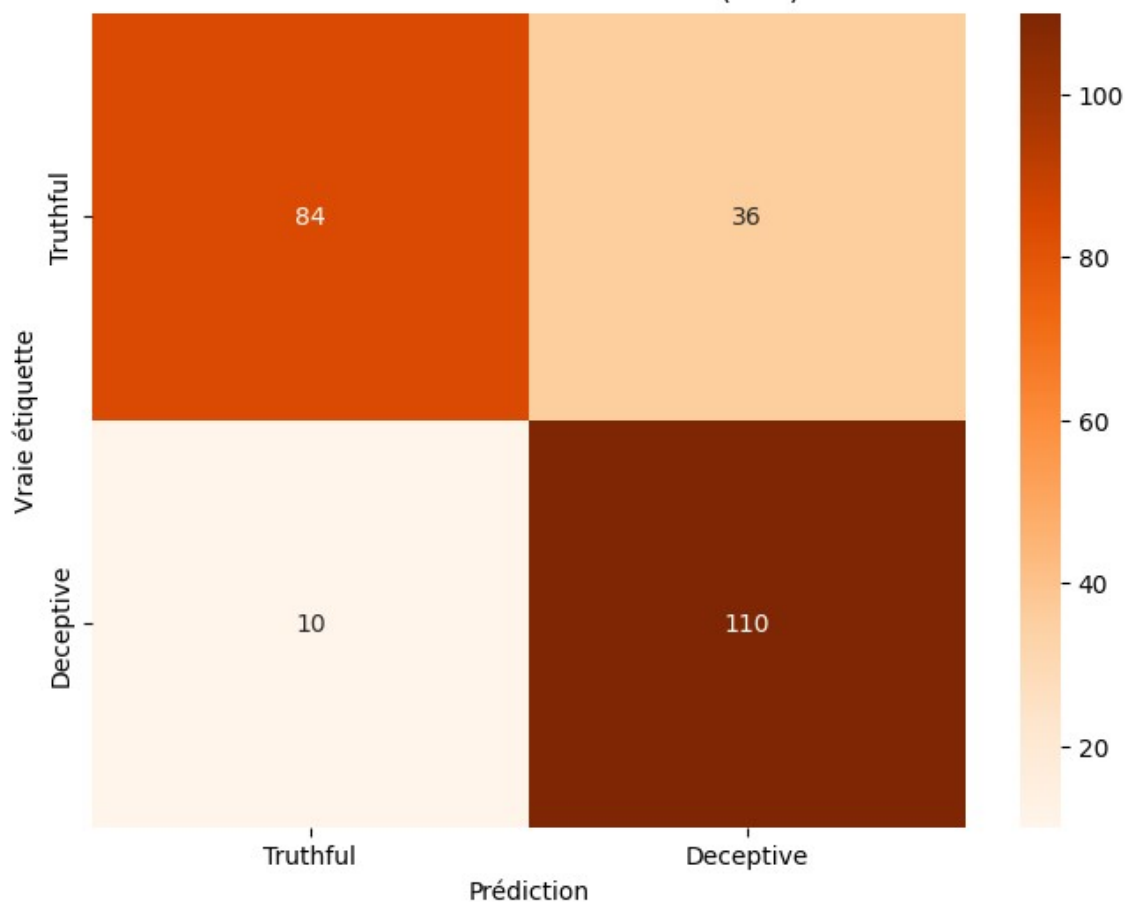
Accuracy: 0.8083

AUC-ROC: 0.9300

Classification Report:

	precision	recall	f1-score	support
Truthful (0)	0.89	0.70	0.79	120
Deceptive (1)	0.75	0.92	0.83	120
accuracy			0.81	240
macro avg	0.82	0.81	0.81	240
weighted avg	0.82	0.81	0.81	240

Matrice de Confusion - Bi-LSTM (Test)



=====

RÉSUMÉ BI-LSTM

Dataset	Accuracy	AUC-ROC
Train	0.8196	0.9268
Validation	0.8333	0.9313
Test	0.8083	0.9300

4. DistilBERT Fine-Tuning Comparison

Now we use a pre-trained Transformer model for comparison.

```
from transformers import DistilBertTokenizer, DistilBertForSequenceClassi

tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")

class BertDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=256):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
            padding="max_length",
            truncation=True,
            return_attention_mask=True,
            return_tensors="pt"
        )
        return {
            "input_ids": encoding["input_ids"].flatten(),
```

```

        "attention_mask": encoding["attention_mask"].flatten(),
        "labels": torch.tensor(self.labels[idx], dtype=torch.long)
    }

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings
You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access

```

warnings.warn(
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 4.98kB/s]

```

```

vocab.txt: 100%  232k/232k [00:00<00:00, 7.91MB/s]

```

```

tokenizer.json: 100%  466k/466k [00:00<00:00, 4.18MB/s]

```

```

def train_bert(model, train_loader, val_loader, optimizer, device, epochs):
    history = {"train_loss": [], "val_acc": []}
    for epoch in range(epochs):
        model.train()
        losses = []
        for batch in tqdm(train_loader):
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
            loss = outputs.loss
            losses.append(loss.item())

            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

        model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for batch in val_loader:
                input_ids = batch["input_ids"].to(device)
                attention_mask = batch["attention_mask"].to(device)
                labels = batch["labels"].to(device)

                outputs = model(input_ids, attention_mask=attention_mask)
                _, preds = torch.max(outputs.logits, dim=1)
                correct += (preds == labels).sum().item()
                total += labels.size(0)

        history["train_loss"].append(np.mean(losses))
        history["val_acc"].append(correct / total)
        print(f"Epoch {epoch+1} - Loss: {history['train_loss'][-1]:.4f} -")
    return history

```

```
# Séparation entraînement / validation (hold-out) pour DistilBERT
from sklearn.model_selection import train_test_split

# Redefine BertDataset class here to fix the AttributeError
# The original class in 'bert_prep' uses tokenizer.encode_plus, which is
# in newer versions of transformers. The direct call tokenizer(...) is no
class BertDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=256):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        encoding = self.tokenizer(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
            padding="max_length",
            truncation=True,
            return_attention_mask=True,
            return_tensors="pt"
        )
        return {
            "input_ids": encoding["input_ids"].flatten(),
            "attention_mask": encoding["attention_mask"].flatten(),
            "labels": torch.tensor(self.labels[idx], dtype=torch.long)
        }

X_train_text, X_val_text, y_train, y_val = train_test_split(
    df['text'], df['label'], test_size=0.2, random_state=42, stratify=df[
])

# Création des datasets
train_dataset = BertDataset(X_train_text.values, y_train.values, tokenize
val_dataset = BertDataset(X_val_text.values, y_val.values, tokenizer)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)

# Chargement du modèle pré-entraîné
model_bert = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2
).to(device)

optimizer = optim.AdamW(model_bert.parameters(), lr=2e-5)
```

```
# Entraînement
bert_history = train_bert(model_bert, train_loader, val_loader, optimizer)

print(f"Meilleure précision de validation pour DistilBERT : {max(bert_history)}
```

config.json: 100%  483/483 [00:00<00:00, 50.9kB/s]

model.safetensors: 100%  268M/268M [00:02<00:00, 211MB/s]

Loading weights: 100% 100/100 [00:00<00:00, 457.89it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weights, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weights]

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
vocab_transform.bias	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
pre_classifier.weight	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture
- MISSING :those params were newly initialized because missing from the checkpoint

100%  80/80 [00:26<00:00, 3.07it/s]

Epoch 1 - Loss: 0.5237 - Val Acc: 0.8375

100%  80/80 [00:26<00:00, 2.96it/s]

Epoch 2 - Loss: 0.3182 - Val Acc: 0.8094

✓ 5. Performance Comparison & Overfitting Analysis

```
def plot_comparison(fold_results):
    avg_train_acc = np.mean([res["train_acc"] for res in fold_results], axis=0)
    avg_val_acc = np.mean([res["val_acc"] for res in fold_results], axis=0)

    plt.figure(figsize=(10, 5))
    plt.plot(avg_train_acc, label="Avg Train Accuracy")
    plt.plot(avg_val_acc, label="Avg Val Accuracy")
    plt.title("Bi-LSTM Performance: Train vs Validation (Overfitting analysis)")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.show()

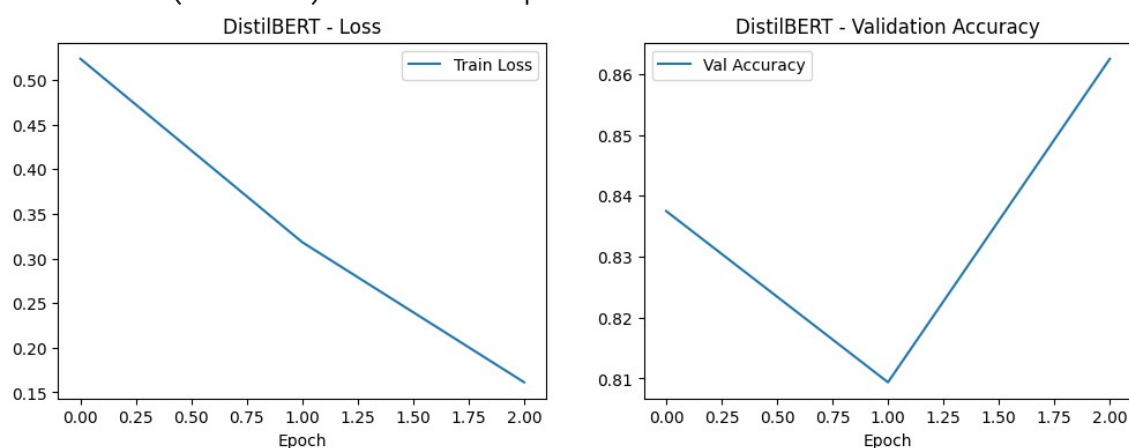
# If train accuracy >> val accuracy, we have overfitting.
# If both are low, we have underfitting.
```

```
# Moyenne des performances du Bi-LSTM sur les 5 folds
avg_bilstm_val_acc = np.mean([max(res['val_acc']) for res in fold_results])
print(f"Bi-LSTM (validation croisée) : précision moyenne = {avg_bilstm_val_acc}")
print(f"DistilBERT (hold-out) : meilleure précision de validation = {max(

# Tracer l'évolution de la perte et de la précision pour DistilBERT
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(bert_history['train_loss'], label='Train Loss')
plt.title("DistilBERT - Loss")
plt.xlabel("Epoch")
plt.legend()

plt.subplot(1,2,2)
plt.plot(bert_history['val_acc'], label='Val Accuracy')
plt.title("DistilBERT - Validation Accuracy")
plt.xlabel("Epoch")
plt.legend()
plt.show()
```

Bi-LSTM (validation croisée) : précision moyenne = 0.7825
 DistilBERT (hold-out) : meilleure précision de validation = 0.8625



```
from sklearn.metrics import classification_report, confusion_matrix, accu
import seaborn as sns
import matplotlib.pyplot as plt

def evaluate_bert_model(model, data_loader, dataset_name="Validation"):
    """Évalue le modèle et retourne les métriques détaillées"""
    model.eval()
    all_preds = []
```

```

all_labels = []
all_probs = []

with torch.no_grad():
    for batch in data_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(input_ids, attention_mask=attention_mask)
        probs = torch.softmax(outputs.logits, dim=1)
        _, preds = torch.max(outputs.logits, dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy()) # Probabilité po

# Calcul des métriques
accuracy = accuracy_score(all_labels, all_preds)
auc_roc = roc_auc_score(all_labels, all_probs)

print(f"\n=== Métriques {dataset_name} - DistilBERT ===")
print(f"Accuracy: {accuracy:.4f}")
print(f"AUC-ROC: {auc_roc:.4f}")
print("\nClassification Report:")
print(classification_report(all_labels, all_preds,
                            target_names=['Truthful (0)', 'Deceptive (1)']))

# Matrice de confusion
cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Truthful', 'Deceptive'],
            yticklabels=['Truthful', 'Deceptive'])
plt.title(f'Matrice de Confusion - DistilBERT ({dataset_name})')
plt.ylabel('Vraie étiquette')
plt.xlabel('Prédiction')
plt.show()

return {
    'accuracy': accuracy,
    'auc_roc': auc_roc,
    'predictions': all_preds,
    'labels': all_labels,
    'probabilities': all_probs,
    'confusion_matrix': cm
}

# Évaluation sur train, validation et test
print("=" * 50)
print("ÉVALUATION DISTILBERT")
print("=" * 50)

# Note: Vous devez créer un DataLoader pour l'entraînement aussi

```

```

train_loader_eval = DataLoader(train_dataset, batch_size=16)
val_loader_eval = DataLoader(val_dataset, batch_size=16)

# Si vous avez un ensemble de test séparé, sinon utilisez validation
# test_loader_eval = DataLoader(test_dataset, batch_size=16)

# Évaluation
train_metrics_bert = evaluate_bert_model(model_bert, train_loader_eval, "
val_metrics_bert = evaluate_bert_model(model_bert, val_loader_eval, "Vali
# test_metrics_bert = evaluate_bert_model(model_bert, test_loader_eval, "

# Résumé comparatif
print("\n" + "=" * 50)
print("RÉSUMÉ DISTILBERT")
print("=" * 50)
print(f"{'Dataset':<15} {'Accuracy':<12} {'AUC-ROC':<12}")
print("-" * 39)
print(f"{'Train':<15} {train_metrics_bert['accuracy']:.4f}           {train_m
print(f"{'Validation':<15} {val_metrics_bert['accuracy']:.4f}           {val_

```

```

=====
ÉVALUATION DISTILBERT
=====

```

```

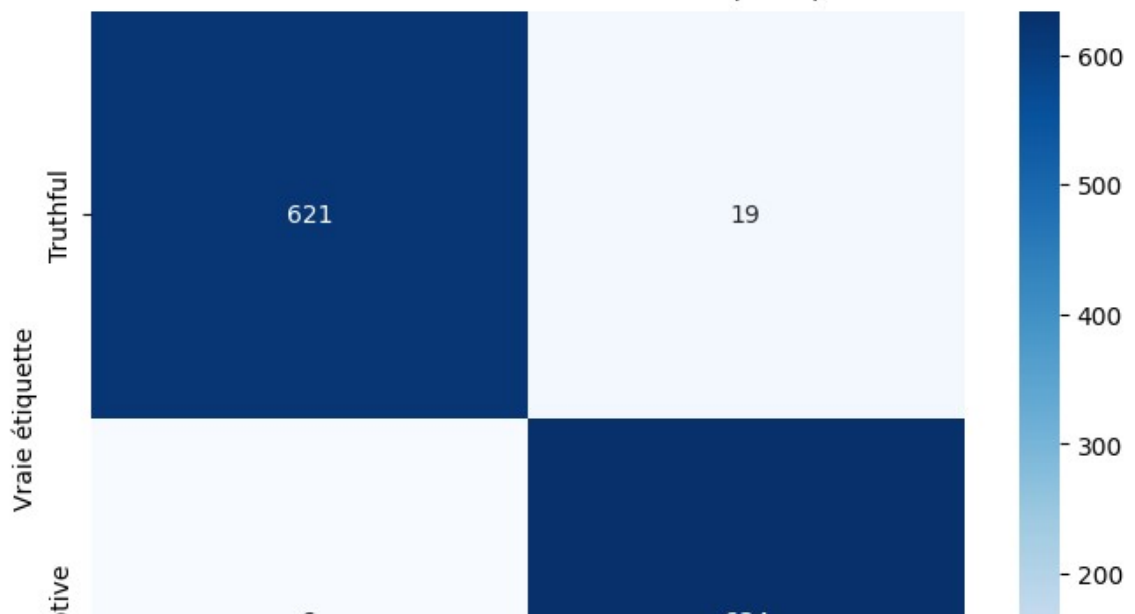
=== Métriques Train - DistilBERT ===
Accuracy: 0.9805
AUC-ROC: 0.9980

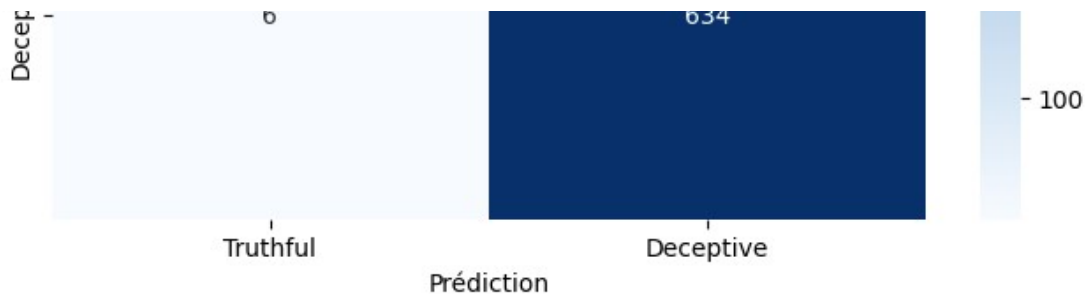
```

Classification Report:

	precision	recall	f1-score	support
Truthful (0)	0.99	0.97	0.98	640
Deceptive (1)	0.97	0.99	0.98	640
accuracy			0.98	1280
macro avg	0.98	0.98	0.98	1280
weighted avg	0.98	0.98	0.98	1280

Matrice de Confusion - DistilBERT (Train)





=== Métriques Validation - DistilBERT ===

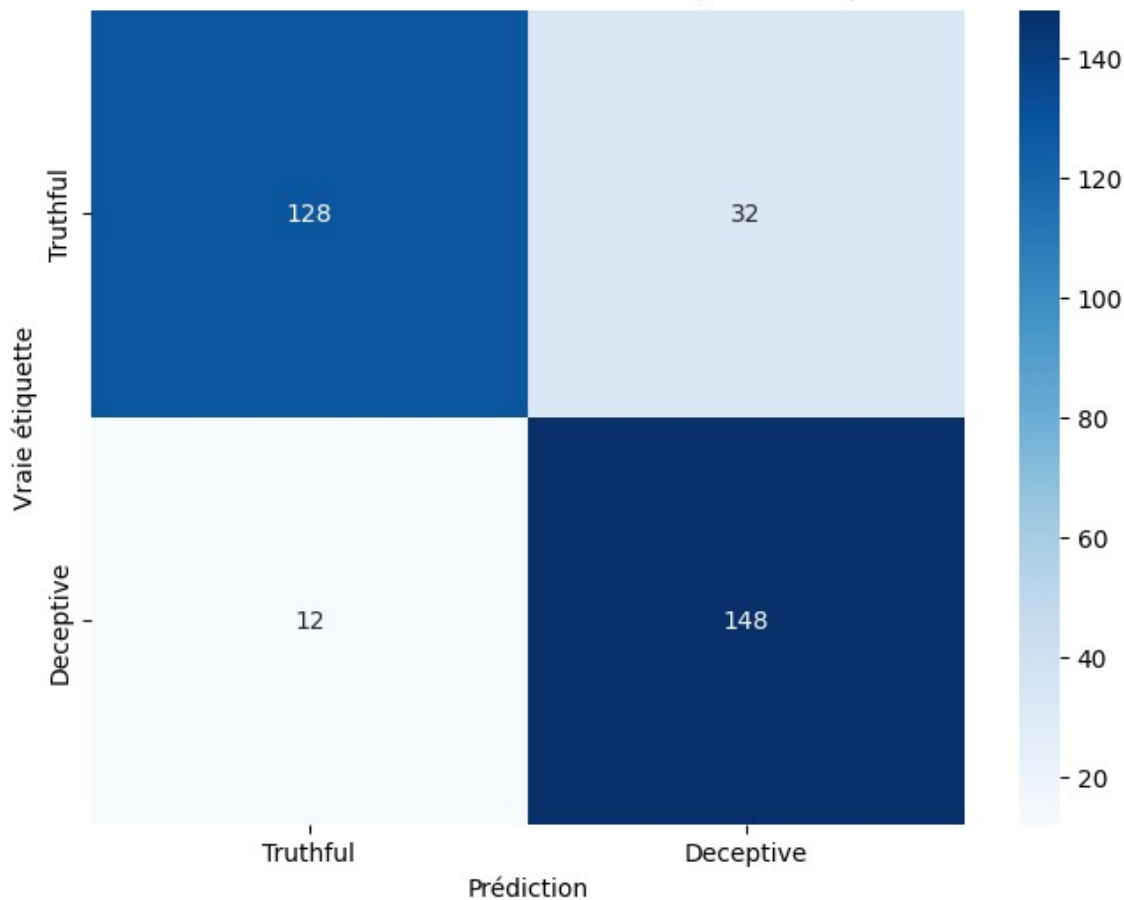
Accuracy: 0.8625

AUC-ROC: 0.9519

Classification Report:

	precision	recall	f1-score	support
Truthful (0)	0.91	0.80	0.85	160
Deceptive (1)	0.82	0.93	0.87	160
accuracy			0.86	320
macro avg	0.87	0.86	0.86	320
weighted avg	0.87	0.86	0.86	320

Matrice de Confusion - DistilBERT (Validation)



=====

RÉSUMÉ DISTILBERT

=====

Dataset	Accuracy	AUC-ROC
Train	0.9805	0.9980

Validation	0.8625	0.9519
------------	--------	--------

```
# Comparaison des performances
print("\n" + "=" * 60)
print("COMPARAISON FINALE DES MODÈLES")
print("=" * 60)

models_comparison = pd.DataFrame({
    'Modèle': ['Bi-LSTM', 'DistilBERT'],
    'Train Accuracy': [train_metrics_lstm['accuracy'], train_metrics_bert],
    'Validation Accuracy': [val_metrics_lstm['accuracy'], val_metrics_bert],
    'Test Accuracy': [test_metrics_lstm['accuracy'], 'N/A'], # DistilBERT
    'Train AUC-ROC': [train_metrics_lstm['auc_roc'], train_metrics_bert['auc_roc']],
    'Validation AUC-ROC': [val_metrics_lstm['auc_roc'], val_metrics_bert['auc_roc']]
})

display(models_comparison)

# Graphique de comparaison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Accuracy comparison
datasets = ['Train', 'Validation']
x = np.arange(len(datasets))
width = 0.35

axes[0].bar(x - width/2,
            [models_comparison.loc[0, 'Train Accuracy'], models_comparison.loc[1, 'Train Accuracy']],
            width, label='Bi-LSTM', color='orange', alpha=0.8)
axes[0].bar(x + width/2,
            [models_comparison.loc[0, 'Validation Accuracy'], models_comparison.loc[1, 'Validation Accuracy']],
            width, label='DistilBERT', color='blue', alpha=0.8)
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Comparaison des Accuracy')
axes[0].set_xticks(x)
axes[0].set_xticklabels(datasets)
axes[0].legend()
axes[0].set_ylim([0, 1])

# AUC-ROC comparison
axes[1].bar(x - width/2,
            [models_comparison.loc[0, 'Train AUC-ROC'], models_comparison.loc[1, 'Train AUC-ROC']],
            width, label='Bi-LSTM', color='orange', alpha=0.8)
axes[1].bar(x + width/2,
            [models_comparison.loc[0, 'Validation AUC-ROC'], models_comparison.loc[1, 'Validation AUC-ROC']],
            width, label='DistilBERT', color='blue', alpha=0.8)
```

```

axes[1].set_ylabel('AUC-ROC')
axes[1].set_title('Comparaison des AUC-ROC')
axes[1].set_xticks(x)
axes[1].set_xticklabels(datasets)
axes[1].legend()
axes[1].set_ylim([0, 1])

plt.tight_layout()
plt.show()

# Analyse du surapprentissage
print("\n" + "=" * 60)
print("ANALYSE DU SURAPPRENTISSAGE")
print("=" * 60)

for model_name, train_acc, val_acc in [
    ('Bi-LSTM', train_metrics_lstm['accuracy'], val_metrics_lstm['accuracy']),
    ('DistilBERT', train_metrics_bert['accuracy'], val_metrics_bert['accuracy']):
]:
    gap = train_acc - val_acc
    print(f"\n{model_name}:")
    print(f"  Train Accuracy: {train_acc:.4f}")
    print(f"  Val Accuracy: {val_acc:.4f}")
    print(f"  Écart Train-Val: {gap:.4f}")
    if gap > 0.1:
        print(f"    ⚠ Surapprentissage détecté (écart > 0.1)")
    elif gap < -0.05:
        print(f"    ⚠ Sous-apprentissage possible (val > train)")
    else:
        print(f"    ✅ Bon équilibre")

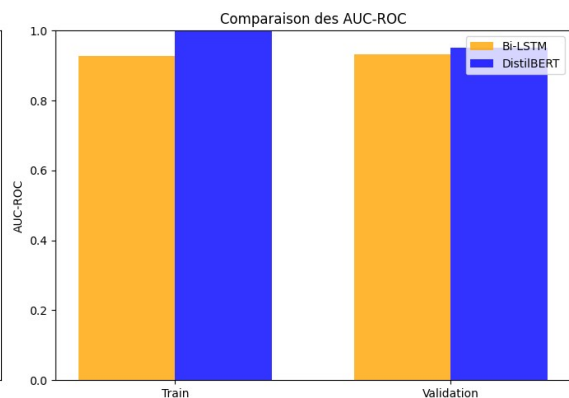
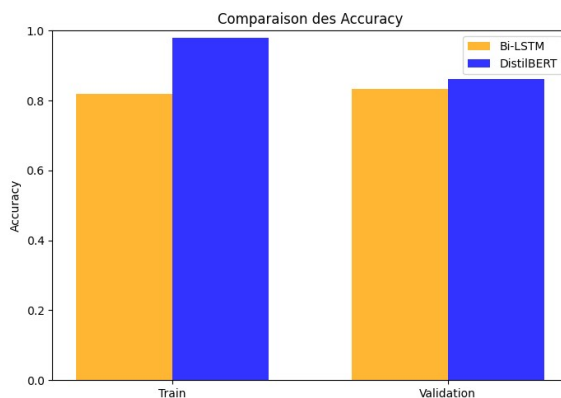
```

```

=====
COMPARAISON FINALE DES MODÈLES
=====

```

	Modèle	Train Accuracy	Validation Accuracy	Test Accuracy	Train AUC-ROC	Validation AUC-ROC
0	Bi-LSTM	0.819643	0.833333	0.808333	0.926767	0.931319
1	DistilBERT	0.980469	0.862500	N/A	0.998005	0.951875



```

=====
ANALYSE DU SURAPPRENTISSAGE
=====

```

```
=====

Bi-LSTM:
  Train Accuracy: 0.8196
  Val Accuracy: 0.8333
  Écart Train-Val: -0.0137
  ✅ Bon équilibre

DistilBERT:
  Train Accuracy: 0.9805
  Val Accuracy: 0.8625
  Écart Train-Val: 0.1180
  ⚠ Surapprentissage détecté (écart > 0.1)
```

Étapes
suivantes :

[Générer du code avec models_comparison](#)[New interactive sheet](#)

✓ 6. Interactive Interface

Finally, we'll create a simple interface to test the models.

```
# Cellule de sauvegarde des modèles entraînés
import os
import torch
import pickle
from datetime import datetime

print(" SAUVEGARDE DES MODÈLES ENTRAÎNÉS")
print("="*60)

# Créer un dossier pour sauvegarder les modèles
save_dir = "saved_models"
os.makedirs(save_dir, exist_ok=True)

# Timestamp pour versionner
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

# 1. SAUVEGARDE DU MODÈLE BI-LSTM
```

```
print("\n Sauvegarde du modèle Bi-LSTM...")

# Vérifier que le modèle existe
if 'model_lstm_final' in locals() or 'model_lstm_final' in globals():
    # Sauvegarder l'état du modèle
    lstm_model_path = os.path.join(save_dir, f"bilstm_model_{timestamp}.pt")
    torch.save({
        'model_state_dict': model_lstm_final.state_dict(), # CORRECTION: n
        'vocab_size': len(vocab_to_int),
        'embedding_dim': 100,
        'hidden_dim': 128,
        'output_dim': 1,
        'n_layers': 2,
        'dropout': 0.5,
        'vocab_to_int': vocab_to_int, # Sauvegarder le vocabulaire
    }, lstm_model_path)
    print(f" ✅ Bi-LSTM sauvegardé: {lstm_model_path}")

    # Sauvegarder aussi une copie comme "best" (pour la compatibilité)
    torch.save(model_lstm_final.state_dict(), os.path.join(save_dir, "best_"))
    print(f" Copie sauvegardée: {os.path.join(save_dir, 'best_lstm.pt')}")
else:
    print(" Modèle Bi-LSTM non trouvé (model_lstm_final n'existe pas)")

# 2. SAUVEGARDE DU MODÈLE DISTILBERT
print("\n Sauvegarde du modèle DistilBERT...")

if 'model_bert' in locals() or 'model_bert' in globals():
    if model_bert is not None:
        # Sauvegarder le modèle fine-tuné
        bert_model_path = os.path.join(save_dir, f"distilbert_model_{timestamp}")

        # Sauvegarde avec transformers
        model_bert.save_pretrained(bert_model_path)
        tokenizer.save_pretrained(bert_model_path)

        print(f" DistilBERT sauvegardé: {bert_model_path}")

        # Sauvegarder aussi les métadonnées
        bert_metadata = {
            'model_type': 'distilbert',
            'num_labels': 2,
            'max_length': 256,
            'training_date': timestamp
        }

        with open(os.path.join(bert_model_path, 'metadata.pkl'), 'wb') as f:
            pickle.dump(bert_metadata, f)

        print(f" Métadonnées sauvegardées")
    else:
        print(" DistilBERT est None - sauvegarde ignorée")
else:
    print(" DistilBERT non disponible - sauvegarde ignorée")
```

```
# 3. SAUVEGARDE DES INFORMATIONS COMPLÉMENTAIRES
print("\n Sauvegarde des informations complémentaires...")

# Sauvegarder la configuration complète
config = {
    'vocab_to_int': vocab_to_int,
    'device': str(device),
    'tokenizer_name': 'distilbert-base-uncased' if 'model_bert' in locals()
    'dataset_size': len(df) if 'df' in locals() else None,
    'num_classes': 2,
    'class_names': ['Truthful', 'Deceptive'],
    'bilstm_params': {
        'vocab_size': len(vocab_to_int),
        'embedding_dim': 100,
        'hidden_dim': 128,
        'n_layers': 2,
        'dropout': 0.5
    }
}

config_path = os.path.join(save_dir, f"config_{timestamp}.pkl")
with open(config_path, 'wb') as f:
    pickle.dump(config, f)

print(f"  Configuration sauvegardée: {config_path}")

# 4. CRÉER UN FICHIER README
print("\n Création du fichier README...")

readme_content = f"""
# Modèles entraînés pour la détection d'avis trompeurs

Date d'entraînement: {timestamp}

## Modèles sauvegardés

### Bi-LSTM
- Fichier: `bilstm_model_{timestamp}.pt`
- Architecture:
  - Vocab size: {len(vocab_to_int)}
  - Embedding dim: 100
  - Hidden dim: 128
  - Num layers: 2
  - Dropout: 0.5

### DistilBERT
- Dossier: `distilbert_model_{timestamp}/`
- Modèle de base: distilbert-base-uncased
- Fine-tuné sur 1600 avis d'hôtels

## Métriques de performance
### Bi-LSTM
- Train Accuracy: {train_metrics_lstm['accuracy']:.4f}
```

```

- Validation Accuracy: {val_metrics_lstm['accuracy']:.4f}
- Test Accuracy: {test_metrics_lstm['accuracy']:.4f}
- Train AUC-ROC: {train_metrics_lstm['auc_roc']:.4f}
- Validation AUC-ROC: {val_metrics_lstm['auc_roc']:.4f}
- Test AUC-ROC: {test_metrics_lstm['auc_roc']:.4f}

## Utilisation
Pour charger les modèles, utilisez les fonctions de chargement appropriées.
"""

readme_path = os.path.join(save_dir, f"README_{timestamp}.txt")
with open(readme_path, 'w') as f:
    f.write(readme_content)

print(f"  README créé: {readme_path}")

# 5. LISTER TOUS LES FICHIERS SAUVEGARDÉS
print("\n" + "="*60)
print("📁 FICHIERS SAUVEGARDÉS")
print("="*60)

if os.path.exists(save_dir):
    for file in os.listdir(save_dir):
        file_path = os.path.join(save_dir, file)
        if os.path.isfile(file_path):
            size = os.path.getsize(file_path) / (1024*1024) # Taille en ME
            print(f"  • {file:<40} ({size:.2f} MB)")
        else:
            # C'est un dossier (pour DistilBERT)
            print(f"  • {file}/")

print("\n✅ Sauvegarde terminée !")
print(f" Dossier de sauvegarde: {save_dir}/")

```

📁 SAUVEGARDE DES MODÈLES ENTRAÎNÉS

=====

📁 Sauvegarde du modèle Bi-LSTM...

- ✅ Bi-LSTM sauvegardé: saved_models/bilstm_model_20260224_120214.pt
- ✅ Copie sauvegardée: saved_models/best_lstm.pt

📁 Sauvegarde du modèle DistilBERT...

Writing model shards: 100% 1/1 [00:07<00:00, 7.05s/it]

- ✅ DistilBERT sauvegardé: saved_models/distilbert_model_20260224_120214
- ✅ Métadonnées sauvegardées

📁 Sauvegarde des informations complémentaires...

- ✅ Configuration sauvegardée: saved_models/config_20260224_120214.pkl

📁 Création du fichier README...

- ✅ README créé: saved_models/README_20260224_120214.txt

=====

📁 FICHIERS SAUVEGARDÉS

FICHIERS SAUVEGARDES

=====

- best_lstm.pt (6.23 MB)
- README_20260224_120214.txt (0.00 MB)
- distilbert_model_20260224_120214/
- bilstm_model_20260224_120214.pt (6.42 MB)
- config_20260224_120214.pkl (0.12 MB)



Sauvegarde terminée !

Dossier de sauvegarde : saved_model/

pip install plotly gradio

```
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/pyth
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.1
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/pyt
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/pyt
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/di
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/pyth
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.1
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/py
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/di
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/p
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/pyt
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/py
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/p
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/py
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.1
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/py
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/pyth
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/pyth
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/di
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dis
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/py
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12
```

```

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/py
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/pyt
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/pyt
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/p
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dis

```

```

import gradio as gr
import torch
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import os
import pickle
import glob
from transformers import DistilBertTokenizer, DistilBertForSequenceClassifi

# =====
# FONCTIONS DE CHARGEMENT DES MODÈLES
# =====

def load_latest_model(model_dir="saved_models", model_type="bilstm"):
    """Charge le dernier modèle sauvegardé"""

    if model_type == "bilstm":
        # Chercher le fichier .pt le plus récent
        pt_files = glob.glob(os.path.join(model_dir, "bilstm_model_*.pt"))
        if not pt_files:
            # Fallback sur best_lstm.pt
            best_file = os.path.join(model_dir, "best_lstm.pt")
            if os.path.exists(best_file):
                print(f"  Chargement de {best_file}")
                return torch.load(best_file, map_location=device)
            return None

        # Prendre le plus récent
        latest_file = max(pt_files, key=os.path.getctime)
        print(f"  Chargement de {latest_file}")
        return torch.load(latest_file, map_location=device)

    elif model_type == "distilbert":
        # Chercher le dossier DistilBERT le plus récent
        bert_dirs = glob.glob(os.path.join(model_dir, "distilbert_model_*"))
        if not bert_dirs:
            return None

        latest_dir = max(bert_dirs, key=os.path.getctime)
        print(f"  Chargement de {latest_dir}")
        return latest_dir

    return None

```

```

def load_config(model_dir="saved_models"):
    """Charge la configuration la plus récente"""
    config_files = glob.glob(os.path.join(model_dir, "config_*.pkl"))
    if not config_files:
        return None

    latest_config = max(config_files, key=os.path.getctime)
    with open(latest_config, 'rb') as f:
        return pickle.load(f)

# =====
# CHARGEMENT DES MODÈLES
# =====
print("="*60)
print(" CHARGEMENT DES MODÈLES SAUVEGARDÉS")
print("="*60)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f" Device: {device}")

# Charger la configuration
config = load_config()
if config:
    vocab_to_int = config.get('vocab_to_int', {})
    print(f" Vocabulaire chargé: {len(vocab_to_int)} mots")
else:
    print(" ⚠ Configuration non trouvée - utilisation des valeurs par défaut")
    # Définir vocab_to_int par défaut si nécessaire
    vocab_to_int = {"<PAD>": 0}

# 1. CHARGEMENT DU BI-LSTM
print("\n🧠 Chargement du modèle Bi-LSTM...")

# Définir la classe BiLSTMClassifier (si pas déjà définie)
class BiLSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim, num_layers=n_layers,
                 dropout=dropout, batch_first=True):
        super(BiLSTMClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim,
                            hidden_dim,
                            num_layers=num_layers,
                            bidirectional=True,
                            dropout=dropout,
                            batch_first=True)
        self.fc = nn.Linear(hidden_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        lstm_out, (hidden, cell) = self.lstm(embedded)
        hidden = self.dropout(torch.cat((hidden[-2,:,:], hidden[-1,:,:]), dim=0))
        out = self.fc(hidden)
        return self.sigmoid(out)

```

```

# Charger le modèle
lstm_checkpoint = load_latest_model(model_type="bilstm")

if lstm_checkpoint:
    if isinstance(lstm_checkpoint, dict) and 'model_state_dict' in lstm_checkpoint:
        # Nouveau format avec métadonnées
        vocab_size = lstm_checkpoint.get('vocab_size', len(vocab_to_int))
        model_lstm = BiLSTMClassifier(
            vocab_size=vocab_size,
            embedding_dim=lstm_checkpoint.get('embedding_dim', 100),
            hidden_dim=lstm_checkpoint.get('hidden_dim', 128),
            output_dim=lstm_checkpoint.get('output_dim', 1),
            n_layers=lstm_checkpoint.get('n_layers', 2),
            dropout=lstm_checkpoint.get('dropout', 0.5)
        ).to(device)
        model_lstm.load_state_dict(lstm_checkpoint['model_state_dict'])

        # Mettre à jour le vocabulaire si disponible
        if 'vocab_to_int' in lstm_checkpoint:
            vocab_to_int = lstm_checkpoint['vocab_to_int']
    else:
        # Ancien format (juste state_dict)
        model_lstm = BiLSTMClassifier(len(vocab_to_int), 100, 128, 1, 2, 0.5)
        model_lstm.load_state_dict(lstm_checkpoint)

    model_lstm.eval()
    print(" ✅ Bi-LSTM chargé avec succès!")
else:
    print(" ⚠️ Bi-LSTM non trouvé - vérifiez le dossier saved_models/")
    model_lstm = None

# 2. CHARGEMENT DE DISTILBERT
print("\n 📦 Chargement du modèle DistilBERT...")

bert_model_dir = load_latest_model(model_type="distilbert")

if bert_model_dir and os.path.exists(bert_model_dir):
    try:
        tokenizer = DistilBertTokenizer.from_pretrained(bert_model_dir)
        model_bert = DistilBertForSequenceClassification.from_pretrained(bert_model_dir)
        model_bert.eval()
        print(" ✅ DistilBERT chargé avec succès!")
    except Exception as e:
        print(f" ⚠️ Erreur lors du chargement de DistilBERT: {e}")
        tokenizer = None
        model_bert = None
else:
    print(" ⚠️ DistilBERT non trouvé - vérifiez le dossier saved_models/")
    tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")
    model_bert = None

print("\n" + "="*60)
print("✅ CHARGEMENT TERMINÉ")

```

```

print("="*60)

# =====
# FONCTIONS DE PRÉDICTION
# =====

def tokenize_text(text, max_len=200):
    """Tokenisation pour le Bi-LSTM"""
    if not vocab_to_int:
        return [0] * max_len

    # Fonction de nettoyage (à définir si nécessaire)
    def clean_text(t):
        import re
        import string
        t = t.lower()
        t = re.sub(f"[{re.escape(string.punctuation)}]", "", t)
        return t

    text = clean_text(text)
    tokens = [vocab_to_int.get(word, 0) for word in text.split()][:max_len]
    if len(tokens) < max_len:
        tokens = tokens + [0] * (max_len - len(tokens))
    return tokens

def predict_with_lstm(text):
    """Prédiction avec le modèle Bi-LSTM"""
    if model_lstm is None:
        return {
            "prediction": 0,
            "label": "MODEL NOT AVAILABLE",
            "confidence": 0,
            "prob_deceptive": 0,
            "prob_truthful": 0
        }

    tokens = torch.tensor([tokenize_text(text)]).to(device)

    with torch.no_grad():
        prob = model_lstm(tokens).item()
        pred_class = 1 if prob > 0.5 else 0
        confidence = prob if prob > 0.5 else 1 - prob

    return {
        "prediction": pred_class,
        "label": "DECEPTIVE (FAKE)" if pred_class == 1 else "TRUTHFUL (REAL)",
        "confidence": confidence,
        "prob_deceptive": prob,
        "prob_truthful": 1 - prob
    }

def predict_with_bert(text):
    """Prédiction avec le modèle DistilBERT"""
    if model_bert is None or tokenizer is None:

```

```

        return {
            "prediction": 0,
            "label": "MODEL NOT AVAILABLE",
            "confidence": 0,
            "prob_deceptive": 0,
            "prob_truthful": 0
        }

encoding = tokenizer(
    text,
    add_special_tokens=True,
    max_length=256,
    padding="max_length",
    truncation=True,
    return_attention_mask=True,
    return_tensors="pt"
)

input_ids = encoding["input_ids"].to(device)
attention_mask = encoding["attention_mask"].to(device)

with torch.no_grad():
    outputs = model_bert(input_ids, attention_mask=attention_mask)
    probs = torch.softmax(outputs.logits, dim=1)
    prob_deceptive = probs[0][1].item()
    pred_class = torch.argmax(probs, dim=1).item()
    confidence = probs[0][pred_class].item()

    return {
        "prediction": pred_class,
        "label": "DECEPTIVE (FAKE)" if pred_class == 1 else "TRUTHFUL (REAL)",
        "confidence": confidence,
        "prob_deceptive": prob_deceptive,
        "prob_truthful": 1 - prob_deceptive
    }

# =====
# FONCTIONS D'INTERFACE (garder le reste identique)
# =====

def create_confidence_gauge(prob_deceptive, model_name):
    """Crée une jauge de confiance avec Plotly"""
    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=prob_deceptive * 100,
        title={'text': f"Probabilité DECEPTIVE ({model_name})"},
        domain={'x': [0, 1], 'y': [0, 1]},
        gauge={
            'axis': {'range': [0, 100], 'tickwidth': 1},
            'bar': {'color': "darkred" if prob_deceptive > 0.5 else "darkgr"},
            'steps': [
                {'range': [0, 50], 'color': "lightgreen"},
                {'range': [50, 100], 'color': "lightcoral"}
            ],
        },
    ))

```

```

        'threshold': {
            'line': {'color': "black", 'width': 4},
            'thickness': 0.75,
            'value': 50
        }
    })
fig.update_layout(height=300, margin=dict(l=20, r=20, t=50, b=20))
return fig

def predict_review_advanced(text, model_choice):
    """Fonction principale de prédiction avec choix du modèle"""
    if not text or len(text.strip()) == 0:
        return "Veuillez entrer un texte à analyser", None

    if model_choice == "Bi-LSTM":
        result = predict_with_lstm(text)
        model_name = "Bi-LSTM"
    else:
        result = predict_with_bert(text)
        model_name = "DistilBERT"

    result_text = f"""
## Résultat de l'analyse

**Modèle utilisé :** {model_name}
**Prédiction :** {result['label']}
**Confiance :** {result['confidence']*100:.2f}%

### Probabilités détaillées :
- Probabilité TRUTHFUL (réel) : {result['prob_truthful']*100:.2f}%
- Probabilité DECEPTIVE (fake) : {result['prob_deceptive']*100:.2f}%

### Analyse du texte :
**Longueur :** {len(text)} caractères, {len(text.split())} mots
"""

    gauge_fig = create_confidence_gauge(result['prob_deceptive'], model_name)
    return result_text, gauge_fig

def compare_models(text):
    """Compare les prédictions des deux modèles"""
    if not text or len(text.strip()) == 0:
        return "Veuillez entrer un texte à comparer", None

    result_lstm = predict_with_lstm(text)
    result_bert = predict_with_bert(text)

    comparison_text = f"""
## Comparaison des modèles

### Bi-LSTM :
- **Prédiction :** {result_lstm['label']}
- **Confiance :** {result_lstm['confidence']*100:.2f}%

```

```

- Probabilité DECEPTIVE : {result_lstm['prob_deceptive']*100:.2f}%
    """

    if result_bert and result_bert['label'] != "MODEL NOT AVAILABLE":
        agreement = "✅ Les modèles sont d'accord" if result_lstm['predict
        comparison_text += f"""
### DistilBERT :
- **Prédiction** : {result_bert['label']}
- **Confiance** : {result_bert['confidence']*100:.2f}%
- Probabilité DECEPTIVE : {result_bert['prob_deceptive']*100:.2f}%

### Verdict : {agreement}
    """

    fig = make_subplots(rows=1, cols=2,
                        subplot_titles=("Bi-LSTM", "DistilBERT"),
                        specs=[[{'type': 'domain'}, {'type': 'domain'}]])

    fig.add_trace(go.Pie(labels=['Truthful', 'Deceptive'],
                          values=[result_lstm['prob_truthful']*100, resu
                          marker_colors=['lightgreen', 'lightcoral'],
                          name="Bi-LSTM"), 1, 1)

    fig.add_trace(go.Pie(labels=['Truthful', 'Deceptive'],
                          values=[result_bert['prob_truthful']*100, resu
                          marker_colors=['lightgreen', 'lightcoral'],
                          name="DistilBERT"), 1, 2)

    fig.update_layout(height=400, showlegend=False)
    return comparison_text, fig
else:
    comparison_text += "\n⚠️ DistilBERT non disponible"
    return comparison_text, None

# =====
# INTERFACE GRADIO
# =====

with gr.Blocks(title="Détecteur d'avis trompeurs", theme=gr.themes.Soft())
    gr.Markdown("""
    # Détecteur d'avis trompeurs (Opinion Spam Detection)

    Cette application utilise deux modèles de deep learning pour détecter s
    - **Bi-LSTM** : Réseau de neurones récurrent
    - **DistilBERT** : Modèle transformer pré-entraîné

    Entrez un avis d'hôtel ci-dessous pour l'analyser !
    """)

    with gr.Tabs():
        with gr.TabItem("Prédiction simple"):
            with gr.Row():
                with gr.Column(scale=2):
                    text_input = gr.Textbox(

```

```

        lines=5,
        placeholder="Entrez un avis d'hôtel à analyser...",
        label="Texte de l'avis"
    )
    model_selector = gr.Radio(
        choices=["Bi-LSTM", "DistilBERT"],
        value="Bi-LSTM",
        label="Choisir le modèle"
    )
    predict_btn = gr.Button("Analyser", variant="primary")

    with gr.Column(scale=1):
        model_status = gr.JSON(label="État des modèles", value=
            "Bi-LSTM": "✅ Chargé" if model_lstm else "❌ Non",
            "DistilBERT": "✅ Chargé" if model_bert else "❌ Non",
        })

    with gr.Row():
        with gr.Column(scale=1):
            result_output = gr.Markdown(label="Résultat")
        with gr.Column(scale=1):
            gauge_output = gr.Plot(label="Jauge de confiance")

    gr.Examples(
        examples=[
            ["The room was clean but smaller than expected. Bed comfort was good."],
            ["This hotel is ABSOLUTELY PERFECT!!! Best hotel EVER!!!"],
        ],
        inputs=[text_input, model_selector],
        outputs=[result_output, gauge_output],
        fn=predict_review_advanced,
        cache_examples=False
    )

    with gr.TabItem(" Comparaison des modèles"):
        with gr.Row():
            text_compare = gr.Textbox(
                lines=5,
                placeholder="Entrez un avis pour comparer les deux modèles",
                label="Texte à comparer"
            )

        compare_btn = gr.Button("Comparer les modèles", variant="primary")

        with gr.Row():
            comparison_result = gr.Markdown(label="Résultat de la comparaison")

        with gr.Row():
            comparison_plot = gr.Plot(label="Comparaison visuelle")

    predict_btn.click(
        fn=predict_review_advanced,
        inputs=[text_input, model_selector],
        outputs=[result_output, gauge_output]
    )

```

```
)

compare_btn.click(
    fn=compare_models,
    inputs=[text_compare],
    outputs=[comparison_result, comparison_plot]
)

# Lancement de l'interface
if __name__ == "__main__":
    demo.launch(share=False, debug=True)
```

```
import gradio as gr
import torch
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import os

# Charger les modèles une seule fois au démarrage
print("Chargement des modèles...")

# Modèle Bi-LSTM
model_lstm = BiLSTMClassifier(len(vocab_to_int), 100, 128, 1, 2, 0.5).to(device)
if os.path.exists("best_lstm.pt"):
    model_lstm.load_state_dict(torch.load("best_lstm.pt", map_location=device))
model_lstm.eval()
print("✓ Bi-LSTM chargé")

# Modèle DistilBERT
try:
    if 'model_bert' in locals() or 'model_bert' in globals():
        model_bert.eval()
        print("✓ DistilBERT chargé")
    else:
        print(" DistilBERT non trouvé, veuillez l'entraîner d'abord")
        model_bert = None
except:
    model_bert = None
    print(" DistilBERT non disponible")

def predict_with_lstm(text):
    """Prédiction avec le modèle Bi-LSTM"""
    tokens = torch.tensor([tokenize_text(text)]).to(device)

    with torch.no_grad():
        prob = model_lstm(tokens).item()
        pred_class = 1 if prob > 0.5 else 0
        confidence = prob if prob > 0.5 else 1 - prob

    return {
        "prediction": pred_class,
        "label": "DECEPTIVE (FAKE)" if pred_class == 1 else "TRUTHFUL (REAL)",
        "confidence": confidence,
        "prob_deceptive": prob,
        "prob_truthful": 1 - prob
    }
```

```

    }

def predict_with_bert(text):
    """Prédiction avec le modèle DistilBERT (version corrigée)"""
    if model_bert is None:
        return {
            "prediction": 0,
            "label": "MODEL NOT AVAILABLE",
            "confidence": 0,
            "prob_deceptive": 0,
            "prob_truthful": 0
        }

    # CORRECTION: utiliser __call__ au lieu de encode_plus
    encoding = tokenizer(
        text,
        add_special_tokens=True,
        max_length=256,
        padding="max_length",
        truncation=True,
        return_attention_mask=True,
        return_tensors="pt"
    )

    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)

    with torch.no_grad():
        outputs = model_bert(input_ids, attention_mask=attention_mask)
        probs = torch.softmax(outputs.logits, dim=1)
        prob_deceptive = probs[0][1].item()
        pred_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0][pred_class].item()

    return {
        "prediction": pred_class,
        "label": "DECEPTIVE (FAKE)" if pred_class == 1 else "TRUTHFUL (REAL)",
        "confidence": confidence,
        "prob_deceptive": prob_deceptive,
        "prob_truthful": 1 - prob_deceptive
    }

def create_confidence_gauge(prob_deceptive, model_name):
    """Crée une jauge de confiance avec Plotly"""
    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=prob_deceptive * 100,
        title={'text': f"Probabilité DECEPTIVE ({model_name})"},
        domain={'x': [0, 1], 'y': [0, 1]},
        gauge={
            'axis': {'range': [0, 100], 'tickwidth': 1},
            'bar': {'color': "darkred" if prob_deceptive > 0.5 else "darkgr"},
            'steps': [
                {'range': [0, 50], 'color': "lightgreen"},

```

```

        {'range': [50, 100], 'color': "lightcoral"}
    ],
    'threshold': {
        'line': {'color': "black", 'width': 4},
        'thickness': 0.75,
        'value': 50
    }
}
))

fig.update_layout(height=300, margin=dict(l=20, r=20, t=50, b=20))
return fig

# CORRECTION: La fonction ne retourne que 2 valeurs (markdown et plot)
def predict_review_advanced(text, model_choice):
    """Fonction principale de prédiction avec choix du modèle"""

    if not text or len(text.strip()) == 0:
        return "Veuillez entrer un texte à analyser", None

    # Prédiction selon le modèle choisi
    if model_choice == "Bi-LSTM":
        result = predict_with_lstm(text)
        model_name = "Bi-LSTM"
    else: # DistilBERT
        result = predict_with_bert(text)
        model_name = "DistilBERT"

    # Création du texte de résultat
    result_text = f"""
## Résultat de l'analyse

**Modèle utilisé :** {model_name}
**Prédiction :** {result['label']}
**Confiance :** {result['confidence']*100:.2f}%

### Probabilités détaillées :
- Probabilité TRUTHFUL (réel) : {result['prob_truthful']*100:.2f}%
- Probabilité DECEPTIVE (fake) : {result['prob_deceptive']*100:.2f}%

### Analyse du texte :
**Longueur :** {len(text)} caractères, {len(text.split())} mots
"""

    # Création de la jauge
    gauge_fig = create_confidence_gauge(result['prob_deceptive'], model_name)

    # CORRECTION: Retourne seulement 2 valeurs
    return result_text, gauge_fig

# CORRECTION: Version simplifiée pour la comparaison
def compare_models(text):
    """Compare les prédictions des deux modèles"""
    if not text or len(text.strip()) == 0:

```

```

        return "Veuillez entrer un texte à comparer", None

    result_lstm = predict_with_lstm(text)
    result_bert = predict_with_bert(text) if model_bert is not None else None

    # Texte comparatif
    comparison_text = f""
    ## Comparaison des modèles

    ### Bi-LSTM :
    - **Prédiction :** {result_lstm['label']}
    - **Confiance :** {result_lstm['confidence']*100:.2f}%
    - Probabilité DECEPTIVE : {result_lstm['prob_deceptive']*100:.2f}%

    ""

    if result_bert and result_bert['label'] != "MODEL NOT AVAILABLE":
        agreement = "✅ Les modèles sont d'accord" if result_lstm['predict'] == result_bert['label'] else "❌ Les modèles ne sont pas d'accord"
        comparison_text += f""

    ### DistilBERT :
    - **Prédiction :** {result_bert['label']}
    - **Confiance :** {result_bert['confidence']*100:.2f}%
    - Probabilité DECEPTIVE : {result_bert['prob_deceptive']*100:.2f}%

    ### Verdict : {agreement}
    ""

    # Graphique comparatif
    fig = make_subplots(rows=1, cols=2,
                        subplot_titles=("Bi-LSTM", "DistilBERT"),
                        specs=[[{'type': 'domain'}, {'type': 'domain'}]])

    fig.add_trace(go.Pie(labels=['Truthful', 'Deceptive'],
                          values=[result_lstm['prob_truthful']*100, result_lstm['prob_deceptive']*100],
                          marker_colors=['lightgreen', 'lightcoral'],
                          name="Bi-LSTM"), 1, 1)

    fig.add_trace(go.Pie(labels=['Truthful', 'Deceptive'],
                          values=[result_bert['prob_truthful']*100, result_bert['prob_deceptive']*100],
                          marker_colors=['lightgreen', 'lightcoral'],
                          name="DistilBERT"), 1, 2)

    fig.update_layout(height=400, showlegend=False)
    return comparison_text, fig
else:
    comparison_text += "\n DistilBERT non disponible"
    return comparison_text, None

# Création de l'interface avancée avec onglets
with gr.Blocks(title="Détecteur d'avis trompeurs", theme=gr.themes.Soft()) as interface:
    gr.Markdown("""
    # Détecteur d'avis trompeurs (Opinion Spam Detection)

    Cette application utilise deux modèles de deep learning pour détecter s

```

- ****Bi-LSTM**** : Réseau de neurones récurrent
- ****DistilBERT**** : Modèle transformer pré-entraîné

Entrez un avis d'hôtel ci-dessous pour l'analyser !
 """)

```
with gr.Tabs():
    with gr.TabItem(" Prédiction simple"):
        with gr.Row():
            with gr.Column(scale=2):
                text_input = gr.Textbox(
                    lines=5,
                    placeholder="Entrez un avis d'hôtel à analyser...",
                    label="Texte de l'avis"
                )
                model_selector = gr.Radio(
                    choices=["Bi-LSTM", "DistilBERT"],
                    value="Bi-LSTM",
                    label="Choisir le modèle"
                )
                predict_btn = gr.Button("Analyser", variant="primary")

            with gr.Column(scale=1):
                model_status = gr.JSON(label="État des modèles", value=
                    "Bi-LSTM": "✅ Chargé",
                    "DistilBERT": "✅ Chargé" if model_bert else "❌ N
                })

        with gr.Row():
            with gr.Column(scale=1):
                result_output = gr.Markdown(label="Résultat")
            with gr.Column(scale=1):
                gauge_output = gr.Plot(label="Jauge de confiance")

        # Exemples préchargés
        gr.Examples(
            examples=[
                ["The room was clean but smaller than expected. Bed con
                ["This hotel is ABSOLUTELY PERFECT!!! Best hotel EVER!!
            ],
            inputs=[text_input, model_selector],
            outputs=[result_output, gauge_output],
            fn=predict_review_advanced,
            cache_examples=False
        )

    with gr.TabItem(" Comparaison des modèles"):
        with gr.Row():
            text_compare = gr.Textbox(
                lines=5,
                placeholder="Entrez un avis pour comparer les deux modè
                label="Texte à comparer"
            )
```

```
compare_btn = gr.Button("Comparer les modèles", variant="primary")

with gr.Row():
    comparison_result = gr.Markdown(label="Résultat de la comparaison")

with gr.Row():
    comparison_plot = gr.Plot(label="Comparaison visuelle")

# Liaison des événements
predict_btn.click(
    fn=predict_review_advanced,
    inputs=[text_input, model_selector],
    outputs=[result_output, gauge_output]
)

compare_btn.click(
    fn=compare_models,
    inputs=[text_compare],
    outputs=[comparison_result, comparison_plot]
)

# Lancement de l'interface
if __name__ == "__main__":
    demo.launch(share=False, debug=True)
```